

Третий этап внешнего курса

Продвинутые темы

Лащиков Алексей Антонович

2026-04-08

Российский университет дружбы народов, Москва, Россия

- Лащиков Алексей Антонович
- Студент группы НКАбд-04-25
- Российский университет дружбы народов им. П. Лумумбы
- 1032253527@rudn.ru
- <https://alexeylashikov.github.io/>



Цель: пройти третий этап внешнего курса «Введение в Linux» и закрепить практические навыки работы в Linux-среде.

Задачи:

- изучить основные команды редактора `vim`;
- освоить написание простых `bash`-скриптов;
- разобрать условия, циклы, функции и рекурсию в `bash`;
- изучить команды `find`, `grep`, `sed`;
- познакомиться с построением графиков в `gnuplot`;
- закрепить работу с правами доступа и файловыми утилитами Linux.

Что изучалось на третьем этапе

В ходе третьего этапа были рассмотрены следующие темы:

- работа с редактором `vim`;
- переменные и аргументы командной строки в `bash`;
- условия, циклы и функции;
- рекурсивные и интерактивные скрипты;
- поиск файлов и обработка текста;
- работа с `gnuplot`;
- права доступа и файловые утилиты.

На начальном этапе были изучены базовые приёмы работы в vim:

- выход из редактора без сохранения;
- различие между word и WORD;
- редактирование строк в normal mode;
- замена текста командой `:%s/.../...`;
- работа в визуальном режиме.

Эти задания были направлены на освоение базовой навигации и редактирования текста.

Примеры заданий по vim

В ходе этапа были выполнены задания на выход из vim, различие между word и WORD, а также редактирование строки в normal mode (Рисунок 1, Рисунок 2, Рисунок 3).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim?
Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Здорово, всё верно.

- ☐ "Esc"
- ☒ ":", затем "q", затем "Enter"
- ☐ "q", затем "Enter"
- ☐ ":", затем "q"
- ☐ "Ctrl", затем "x"

Следующий шаг

Решить снова

Ваши решения Вы получили: 1 балл

Верно решил 47 171 учащихся
Из всех попыток 67% верные

При переключении в vim "то слова" есть небольшая разница в том, используем мы маленькую (a, b) или большую (A, E, B) букву. Первые переводят нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим переключениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, ответьте ниже все верно утверждения по следующей строке:
StranGe, TEXT, La,ngue, 3rd XEE!

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать клетку по, например, переключению ж, нужно открыть vim и ввести команду :<tab>j. Вы попадете в то место строки, где отмечено это переключение, а так как все переключения отмечены рядом, то двигаться по тексту вверх и вниз можно при помощи стрелок вправо и влево. Кроме того, можно вызвать сразу строку по номеру word при помощи :<tab>word. Чтобы закрыть строку, нужно ввести команду :q.

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили задание задум, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свой ответ с другим на [форуме решений](#).

- ☒ После 10 нажатий на W курсор окажется там же, где он был после 10 нажатий на w
- ☒ В этой строке 5 "больших слов" (WORD)
- ☒ В этой строке 5 "слов" (word)
- ☒ В этой строке 9 "большых слов" (WORD)
- ☒ В этой строке 9 "слов" (word)
- ☒ Нажав только на W, нельзя переместить курсор на "

Следующий шаг

Решить снова

Ваши решения Вы получили: 1 балл

Верно решили 26 407 учащихся
Из всех попыток 78% верные

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five  
и вам нужно преобразовать её в строку  
three four four four five
```

Какой(их) из предложенных ниже наборов **нажатий клавиш** вызовет такое редактирование? В этих наборах нажатия на клавишу Esc обозначаются как <Esc> (т.е. знаки "<" и ">" не имеют отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Правильно.

Вы решили задание задум, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свой ответ с другим на [форуме решений](#).

- ☒ dDwuyPr
- ☒ xxxxxxxxuyPr
- ☒ dDySafour four~Esc
- ☒ dDwuyPr
- ☒ ddthree four four four five~Esc

Следующий шаг

Решить снова

Ваши решения Вы получили: 1 балл

Рис. 1: Выбор комбинации клавиш для выхода из vim

Рис. 2: Задание на различие word и WORD в vim

Рис. 3: Редактирование строки в vim с помощью команд normal mode

Следующий блок был посвящён базовым средствам bash.

Были рассмотрены:

- история команд во вложенных оболочках;
- допустимые имена переменных;
- позиционные параметры \$1, \$2;
- вывод аргументов командной строки;
- базовая структура простого скрипта.

Этот раздел показал, как строятся самые простые сценарии автоматизации.

Отдельный раздел был посвящён логике программ на bash.

Были изучены:

- условные конструкции `if`, `elif`, `else`;
- циклы и их поведение;
- арифметические операции;
- локальные и глобальные переменные;
- функции и рекурсивные вызовы.

Эти темы необходимы для создания более сложных и полезных скриптов.

В ходе выполнения заданий было установлено, что:

- условные выражения позволяют строить ветвление программы;
- цикл может менять поведение с помощью `continue`;
- функции в `bash` поддерживают работу с локальными и глобальными переменными;
- рекурсия может использоваться, например, для вычисления НОД;
- интерактивные скрипты позволяют организовать диалог с пользователем.

Практические задания по скриптам

На третьем этапе были написаны и проверены более сложные сценарии, включая функции и рекурсивные вызовы. В частности, был выполнен разбор задания на локальные и глобальные переменные в функции, а также задания по работе с `find` (Рисунок 7, Рисунок 8).

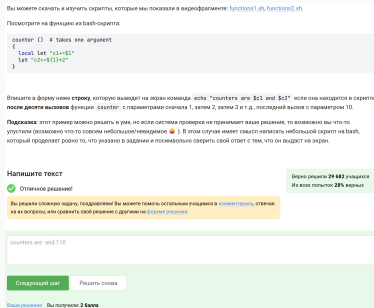


Рис. 7: Задание на локальные и глобальные переменные в функции `bash`

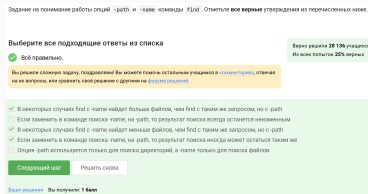


Рис. 8: Понимание различий между `-path` и `-name` в `find`

Большой блок заданий был связан с обработкой файлов и текста в Linux.

Были рассмотрены:

- поиск файлов через `find`;
- различие между `-name`, `-iname`, `-path`;
- использование ограничений глубины поиска;
- регулярные выражения в `grep`;
- выборочный вывод и замена текста через `sed`.

Этот раздел особенно важен для работы с файлами и автоматизации рутинных операций.

На третьем этапе также были изучены базовые возможности gnuplot.

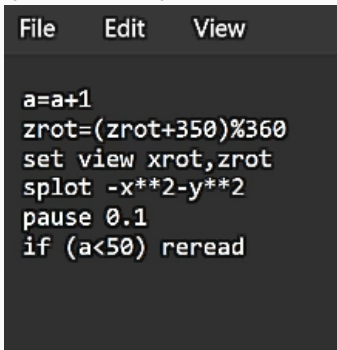
Были рассмотрены:

- запуск программы с сохранением окна после завершения;
- использование заголовков столбцов как названий рядов;
- построение графиков по данным из файла;
- изменение сценария анимации в файле `move.rot`.

Это позволило познакомиться с простейшей визуализацией данных в Linux.

Пример задания по gnuplot

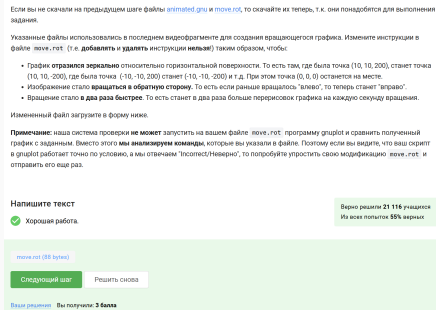
В практической части был изменён файл `move.rot` так, чтобы график зеркально отражался, вращался в обратную сторону и делал это быстрее (Рисунок 9, Рисунок 10).



```
File Edit View

a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Рис. 9: Изменённое содержимое файла `move.rot`



Если вы не скачали на предыдущем шаге файлы `animated.gnu` и `move.rot`, то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять и удалять инструкции нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Изменённый файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Хорошая работа.

Верно решили 21 116 учащихся
Из всех попыток 55% верных

`move.rot` (88 bytes)

Следующий шаг Решить снова

Ваши решения Вы получили: 3 балла

Рис. 10: Проверка задания по изменению `move.rot`

Заключительный блок был посвящён повседневным утилитам Linux.

Были рассмотрены:

- изменение прав доступа через `chmod`;
- права на каталоги и возможность создавать файлы;
- подсчёт строк, слов и символов через `wc`;
- определение размера каталога командой `du`;
- быстрое создание нескольких каталогов через `mkdir`.

Эти команды активно используются в обычной работе в Linux.

В результате прохождения третьего этапа были закреплены навыки:

- редактирования текста в `vim`;
- написания простых и более сложных `bash`-скриптов;
- использования условий, циклов, функций и рекурсии;
- поиска файлов и обработки текста;
- построения графиков в `gnuplot`;
- управления правами доступа и работы с файловыми утилитами.

В ходе выполнения третьего этапа внешнего курса были изучены и закреплены приёмы работы с редактором `vim`, написанием `bash`-скриптов, поиском файлов, обработкой текста, визуализацией данных и управлением правами доступа.

Практические задания позволили не только проверить теоретические знания, но и получить навыки, которые полезны для реальной работы в Linux и автоматизации повседневных задач.